

スクリプトプログラミング演習2

最終課題レポート

Japan Earthquake Monitor
GitHub・ngrok・Jenkins による自動生成

学籍番号 HW25A064

氏名 實光 駿斗

開発環境 macOS

1. 課題の目的

本課題では、GitHub への push をきっかけとして Jenkins が自動実行される仕組みを構築した。テーマは日本周辺の地震情報とし、Python でデータ取得・整形・検証を行い、JavaScript で閲覧用 HTML を生成する。ngrok を利用してローカルの Jenkins へ GitHub Webhook を中継し、ソース変更から成果物生成までを自動化することを目的とした。

2. 作成したシステム

プロジェクト名は「Japan Earthquake Monitor」とした。USGS Earthquake Catalog の GeoJSON データを取得し、日本周辺・過去7日間・マグニチュード 2.5 以上という条件で整理する。外部サービスが一時的に利用できない場合でも検証を継続できるよう、同じ形式のオフライン用サンプルデータも同梱した。

自動処理の流れ

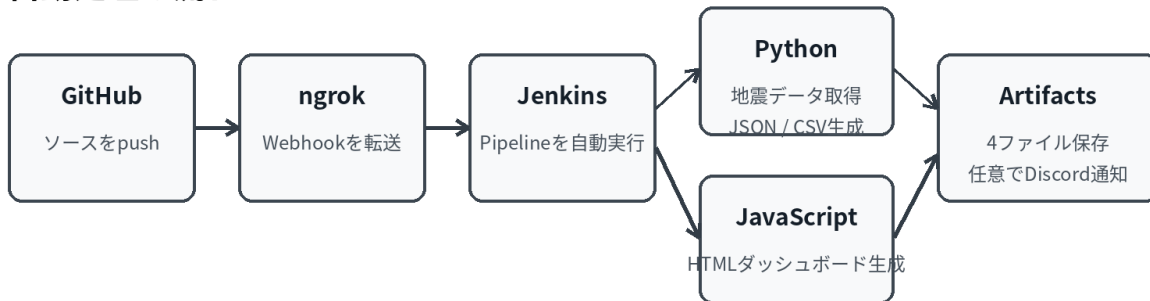


図1 GitHub から成果物生成までの処理の流れ

3. 出力する成果物

ファイル	内容
earthquakes.json	学生情報、取得条件、集計値、地震イベントを保持する JSON
earthquakes.csv	表計算ソフトでも確認できる地震イベント一覧
index.html	検索、最低マグニチュード絞り込み、日別件数グラフを備えた HTML
build_summary.json	ファイル存在、件数整合性、HTML 内容を検査した結果

4. 実装内容

4.1 Python によるデータ処理

earthquake_fetcher.py では、API から取得した GeoJSON を読み取り、発生時刻を UTC と日本時間の両方で保存した。緯度・経度・深さ・マグニチュード・場所・津波フラグを共通形式へ変換し、JSON と CSV へ出力する。件数、最大マグニチュード、平均マグニチュード、平均深さ、M5 以上の件数も同時に計算する。

4.2 JavaScript による表示

build_dashboard.mjs では、Python が作成した JSON を HTML へ埋め込む。外部ライブラリを読み込まないため、Jenkins の成果物として保存した index.html 単体で表示できる。地名検索、最低マグニチュードによる絞り込み、日別件数の棒グラフを実装した。

4.3 エラー対策

API 通信が失敗した場合、--allow-fallback 指定時は同梱 fixture へ切り替える。通信障害を無視して空データを成功扱いにするのではなく、必ず 1 件以上のデータが存在することを確認している。

verify_outputs.py では JSON・CSV・HTML の件数と内容を照合し、不整合があれば Jenkins を失敗させる。

4.4 秘密情報の管理

ngrok authtoken、GitHub トークン、Discord Webhook URL はソースコードへ記載しない。Discord 通知を使用する場合は Jenkins Credentials から環境変数 DISCORD_WEBHOOK_URL として渡す。値が設定されていない場合は通知処理のみを安全に省略する。

5. Jenkins Pipeline

Jenkinsfile では以下の順番で処理する。

1. GitHub リポジトリからソースを Checkout する。
2. Python、Node.js、Git のバージョンを記録する。
3. Python の単体テストを実行する。
4. 地震データを取得し、JSON と CSV を作成する。
5. JavaScript で HTML ダッシュボードを作成する。
6. 4 成果物の整合性を検査する。
7. 成果物を Jenkins Artifacts として保存し、設定されている場合は Discord へ結果を通知する。

GitHub 側では push イベントを Webhook として送信し、ngrok が localhost:8080 の Jenkins へ転送する。Jenkins ジョブでは「GitHub hook trigger for GITScm polling」を有効にし、手動で「ビルド実行」を押さなくても新しいビルドが始まる構成とする。

6. テスト結果

提出用プロジェクト作成時に、オフライン fixture を使用してローカル検証を実施した。結果は次の通りである。

確認項目	結果	補足
Python 単体テスト	6 件すべて成功	正規化、並び順、集計、成果物検査を確認
JSON 出力	成功	12 件のサンプルイベントを保存
CSV 出力	成功	JSON と同じ 12 件を保存
HTML 生成	成功	検索・絞り込み・日別グラフを実装
成果物検査	成功	build_summary.json の status は success

Jenkins 実行動画とコンソールログについては、本人の Mac 上で GitHub・ngrok・Jenkins を実際に連携させた成功ビルドから取得する。最終提出 ZIP 作成スクリプトは、ログに「Finished: SUCCESS」がない場合は処理を中止するため、架空の成功ログを使用できないようにした。

7. 自に追加した要素

- 授業例とは異なる地震情報をテーマにし、JSON・CSV・HTML へ展開した。
- JavaScript で検索、マグニチュード絞り込み、日別件数グラフを実装した。
- API 障害時のオフラインフォールバックを実装した。
- 成果物同士の件数や学生番号まで自動検証する処理を追加した。
- Discord 通知を秘密情報をコードへ保存せずに利用できるようにした。
- Mac 用の最終提出 ZIP 生成スクリプトを作成した。

8. まとめ

GitHub、ngrok、Jenkins を連携させることで、ソースコードを push するだけでテスト、データ取得、HTML 生成、検証、成果物保存までを自動化できる構成を作成した。単に処理を動かすだけでなく、外部通信の失敗や出力の不整合を検出する仕組みも追加した。今回の課題を通じて、プログラム本体だけでなく、変更を安全に確認して成果物を作る CI の重要性を理解できた。

付 主要ファイル

パス	役割
Jenkinsfile	Jenkins Pipeline 定義
src/earthquake_fetcher.py	地震データ取得・整形・JSON/CSV 出力
src/build_dashboard.mjs	HTML ダッシュボード生成
src/verify_outputs.py	成果物検査
src/notify_discord.py	任意の Discord 通知
tests/	Python 単体テスト
submission/make_submission.sh	動画・ログを含む最終 ZIP 生成